

Case Eventos

Documentacao Tecnica Geral do Projeto

Aplicacao fullstack para gerenciamento de eventos e participantes, construida com Node.js, TypeScript, PostgreSQL, Express, Prisma, Next.js, React, Axios, Docker e CI/CD.

Autor: Cledson Santos

Empresa: HUMU

Data de geracao: 09/06/2026

Versao: 1.0.0

Introducao e Contexto

Objetivo deste capitulo

Este capitulo apresenta o projeto Case Eventos como entrega fullstack para o teste tecnico. A ideia e contextualizar o problema, a empresa, o escopo da solucao e como a documentacao geral se conecta aos materiais especificos de backend e frontend.

Contexto do desafio

O teste pede uma aplicacao fullstack simples para gerenciar eventos e participantes. A proposta nao e medir volume de funcionalidades, mas avaliar:

- familiaridade com a stack;
- organizacao de codigo;

- clareza de implementação;
- uso correto de TypeScript;
- separação de responsabilidades;
- migrations;
- versionamento com Git;
- frontend funcional e navegável.

Por isso, a entrega foi planejada como uma aplicação pequena, mas completa: API REST, banco relacional, cache opcional, frontend navegável, Docker, CI/CD, testes e documentação.

Contexto da HUMU

A HUMU se posiciona como uma estrutura para organizar operações financeiras fragmentadas. A proposta da empresa fala sobre integrar pagamentos, conta digital, crédito, infraestrutura financeira, gestão de ativos e energia dentro de uma arquitetura única.

Mesmo sendo um case de eventos, a entrega foi construída com esse mesmo critério de organização:

- componentes separados por responsabilidade;
- backend e frontend independentes, mas integrados;
- contratos claros entre camadas;
- infraestrutura reproduzível;
- deploy com ambientes;
- documentação para operação e avaliação.

A frase "quatro frentes, uma arquitetura" serviu como referência de leitura: o projeto também separa frentes diferentes, mas mantém uma arquitetura coesa.

Proposta da solução

O Case Eventos permite:

- criar eventos;
- criar participantes;
- inscrever participantes em eventos;

- listar participantes de um evento;
- buscar e paginar dados;
- navegar por uma interface web;
- consultar contratos da API via Swagger.

O projeto foi organizado como monorepo:

```
case-eventos/ backend/ frontend/ deploy/ scripts/ docs/
```

O que esta documentação cobre

Esta documentação geral cobre a visão fullstack:

- aderência ao teste;
- arquitetura completa;
- relação entre backend, frontend, banco, cache e deploy;
- comandos de execução;
- CI/CD;
- qualidade, testes e performance;
- decisões técnicas e tradeoffs.

Documentações específicas:

- **backend/docs**: detalhes internos da API;
- **frontend/docs**: detalhes internos da interface;
- **README.md**: instruções resumidas de setup e entrega.

Leitura recomendada para avaliação

Para uma avaliação rápida:

1. Leia **02-aderencia-ao-desafio.md**.
2. Leia **04-arquitetura-fullstack.md**.
3. Rode o projeto seguindo **09-execucao-local-e-docker.md**.

4. Consulte [11-testes-qualidade-e-performance.md](#).
5. Abra [backend/docs](#) e [frontend/docs](#) para aprofundar.

Resultado esperado

Ao final da leitura, o avaliador deve conseguir responder:

- quais requisitos do teste foram atendidos;
- como o projeto foi organizado;
- como rodar localmente;
- como validar a entrega;
- onde cada responsabilidade está implementada;
- quais escolhas demonstram maturidade técnica.

Aderência ao Desafio

Objetivo deste capítulo

Este capítulo mapeia os requisitos do teste técnico para a implementação entregue. A ideia é facilitar a leitura do avaliador e mostrar onde cada item foi atendido.

Back-end

Stack pedida

Requisito	Entrega
Node.js	Backend em Node.js 22
TypeScript	TypeScript strict
PostgreSQL	PostgreSQL via Prisma

Express.js	Express 5
ORM	Prisma 7
Migrations	backend/prisma/migrations

Rotas obrigatorias

Rota pedida	Status	Implementacao
POST /events	Feita	Cria evento
POST /participants	Feita	Cria participante
POST /events/:eventId/participants	Feita	Inscribe participante em evento
GET /events/:eventId/participants	Feita	Lista participantes do evento

Rotas complementares

Rota	Motivo
GET /events	Necessaria para listagem no frontend
GET /events/:eventId	Necessaria para detalhe no frontend
DELETE /events/:eventId	Administracao basica de eventos
GET /participants	Busca por e-mail e listagem
DELETE /participants/:participantId	Remocao de participantes
/health, /livez, /readyz	Operacao e deploy
/docs, /docs.json	Swagger/OpenAPI

Front-end

Stack pedida

Requisito	Entrega
React	React 19
TypeScript	TypeScript strict
Next.js	Next.js 16 com App Router
Axios	Axios no cliente HTTP e Route Handlers
Redux ou Context opcional	Context API para toasts
Sem biblioteca de UI	Componentes próprios com Tailwind CSS v4

Telas obrigatorias

Tela pedida	Status	Implementacao
Pagina de listagem de eventos	Feita	<code>/events</code>
Pagina de detalhes de evento com participantes	Feita	<code>/events/:eventId</code>
Formulario de criacao de eventos	Feita	<code>/events/new</code>
Formulario de inscricao de participantes	Feita	detalhe do evento

Critérios de avaliação

Organização e estrutura

O projeto e organizado como monorepo:

```
backend/ frontend/ deploy/ scripts/ docs/
```

Cada parte possui responsabilidade propria e documentacao tecnica separada.

Clareza na implementacao

O backend separa:

- routes;
- controllers;
- services;
- repositories;
- schemas;
- docs;
- infrastructure;
- shared.

O frontend separa:

- app routes;
- components;
- services;
- lib;
- hooks;
- utils;
- types.

TypeScript

TypeScript e usado de ponta a ponta:

- DTOs e schemas no backend;
- tipos de entidades e responses no frontend;

- validação de envs;
- paginação tipada;
- services tipados.

Separacao de responsabilidades

O backend evita regra de negocio em controller e acesso a banco fora de repository. O frontend evita expor token ao browser e usa Route Handlers como BFF para escritas.

Git

O trabalho foi feito em branch `development`, com commits em portugues e no padrao Conventional Commits.

Aplicacao funcional

A aplicacao roda localmente, via Docker e em deploy por VPS. O fluxo completo foi validado:

1. criar evento;
2. criar participante;
3. inscrever em evento;
4. listar participantes;
5. buscar participantes;
6. paginar resultados.

Complementos tecnicos

Alem do pedido minimo, o projeto inclui:

- Swagger/OpenAPI;
- Redis cache opcional;
- seed com evento de 1000 participantes;
- testes unitarios, HTTP, banco e frontend;
- scripts de performance;
- Dockerfiles;

- Docker Compose;
- CI e deploy via GitHub Actions;
- documentacao tecnica completa.

Conclusao

O projeto atende o escopo obrigatorio do teste e adiciona complementos proporcionais para demonstrar maturidade tecnica sem fugir do objetivo: entregar uma aplicacao fullstack simples, organizada, tipada e funcional.

Visao Geral da Solucao

Objetivo deste capitulo

Este capitulo descreve a solucao completa em alto nivel: dominios, fluxos, componentes principais e responsabilidades de cada parte.

Dominios

O sistema trabalha com tres conceitos principais:

- evento;
- participante;
- inscricao de participante em evento.

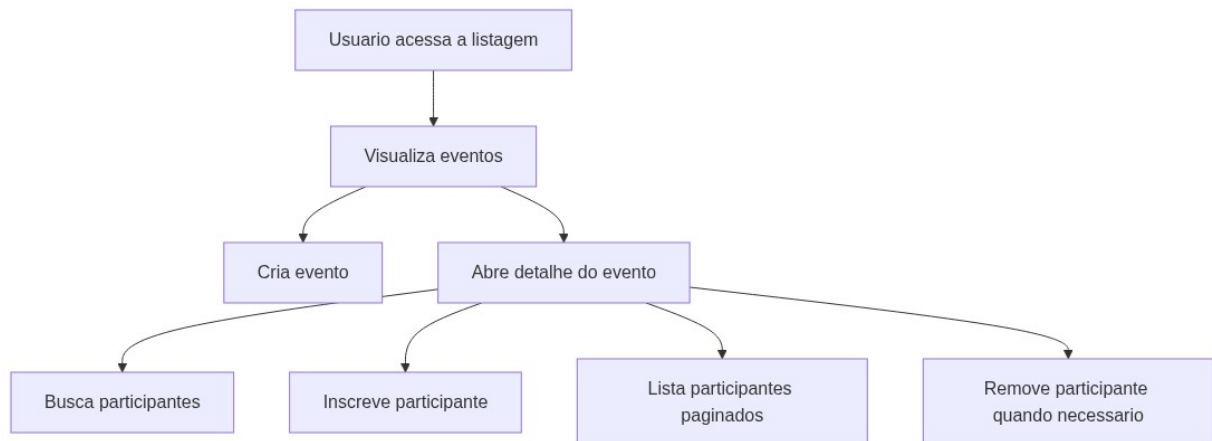
No backend, esses conceitos sao modelados como:

- `Event`;
- `Participant`;
- `EventParticipant`.

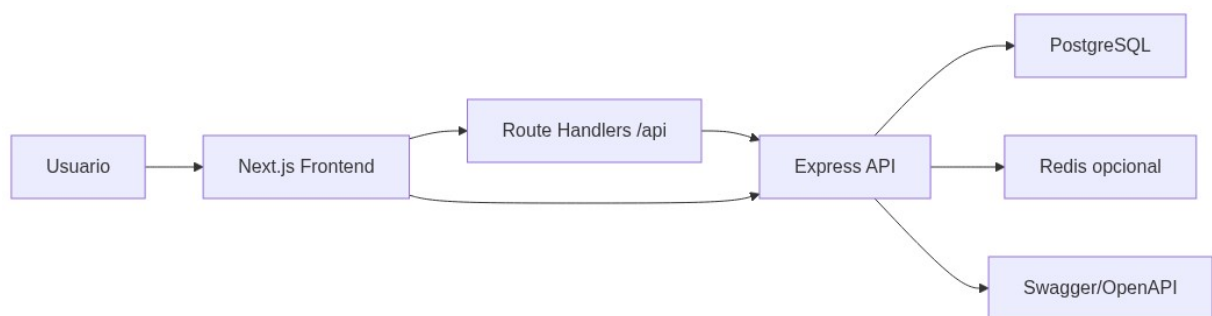
No frontend, os mesmos conceitos aparecem como tipos em `frontend/src/types`.

Fluxo funcional

O fluxo principal é:



Componentes de sistema



Backend

O backend expoe API REST protegida por Bearer token nas rotas de dominio. Ele usa Express, Prisma, PostgreSQL, Redis opcional, Swagger, Pino, Zod e Vitest.

Responsabilidades:

- validar entradas;
- aplicar regras de negocio;
- persistir dados;
- invalidar cache;
- responder em formato padronizado;
- documentar contratos;
- fornecer health checks.

Frontend

O frontend entrega a interface web com Next.js, React, TypeScript, Tailwind CSS e Axios.

Responsabilidades:

- listar eventos;
- exibir detalhes;
- enviar formulários;
- mostrar feedback;
- manter token fora do browser;
- tratar loading, erro e vazio;
- preservar filtros e paginação.

Banco

PostgreSQL armazena:

- eventos;
- participantes;
- relação entre evento e participante.

A relação de inscrição usa chave composta para impedir duplicidade no nível do banco.

Cache

Redis é usado como cache opcional para leituras. Se Redis estiver indisponível, a API continua funcionando sem cache.

Essa escolha melhora resiliência e evita que uma dependência auxiliar derrube o fluxo principal.

Deploy

O projeto possui Dockerfile para backend e frontend, compose local e compose de VPS.

O deploy usa:

- GitHub Actions;
- GHCR;
- SSH na VPS;
- Docker Compose;
- nginx;
- certbot.

Documentacao

As documentacoes estao separadas por nivel:

```
docs/ documentacao geral fullstack backend/docs/ documentacao tecnica do backend  
frontend/docs/ documentacao tecnica do frontend README.md setup resumido
```

Resultado

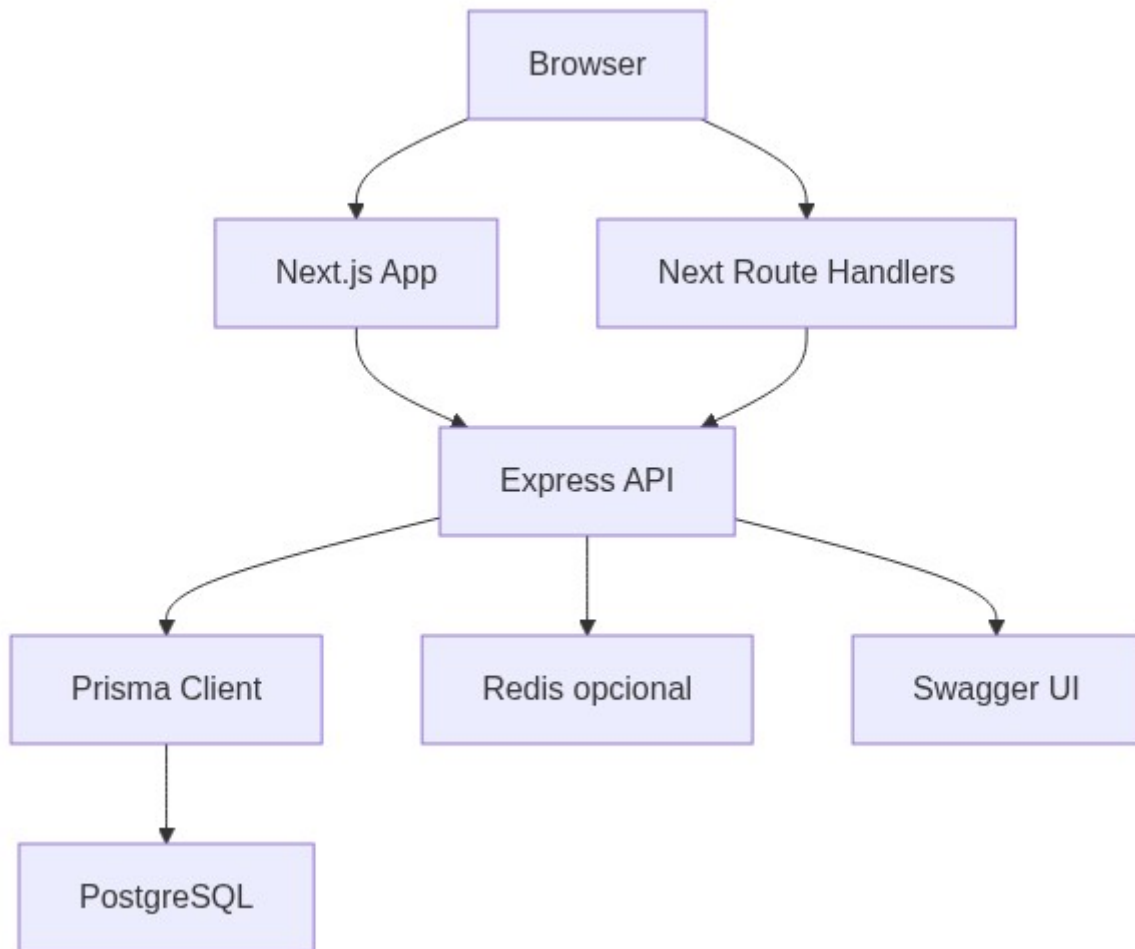
A solucao entrega uma experiencia completa: o avaliador pode rodar o projeto, testar a API no Swagger, navegar no frontend, ver o banco com dados seedados, executar testes e entender a arquitetura pelas documentacoes.

Arquitetura Fullstack

Objetivo deste capitulo

Este capitulo detalha a arquitetura fullstack do projeto e como backend, frontend, banco, cache e deploy se conectam.

Visao geral



Separacao entre frontend e backend

Backend e frontend ficam em pastas independentes:

```
backend/ frontend/
```

Cada aplicacao possui:

- `package.json`;
- scripts proprios;
- Dockerfile proprio;
- `.env.example`;
- documentacao tecnica propria.

Essa separacao permite rodar apenas uma parte quando necessario, mas tambem subir tudo pelo compose da raiz.

Contrato HTTP

O frontend consome o backend via HTTP. O backend responde com envelope padronizado:

```
{ "success": true, "message": "Operacao realizada com sucesso", "data": {},  
  "timestamp": "2026-06-09T12:00:00.000Z" }
```

No frontend, `httpClient` remove o envelope e entrega apenas `data` aos services.

Seguranca do token

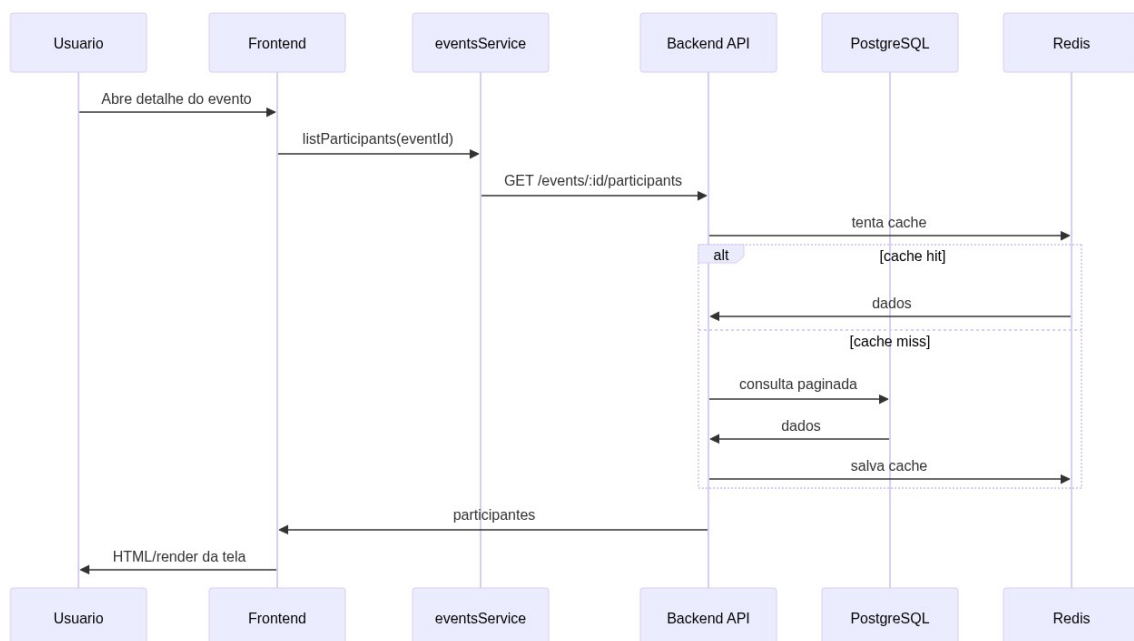
O backend protege rotas de dominio com Bearer token.

O frontend não expõe esse token ao browser. Leituras server-side e Route Handlers internos injetam o header:

```
Authorization: Bearer <API_TOKEN>
```

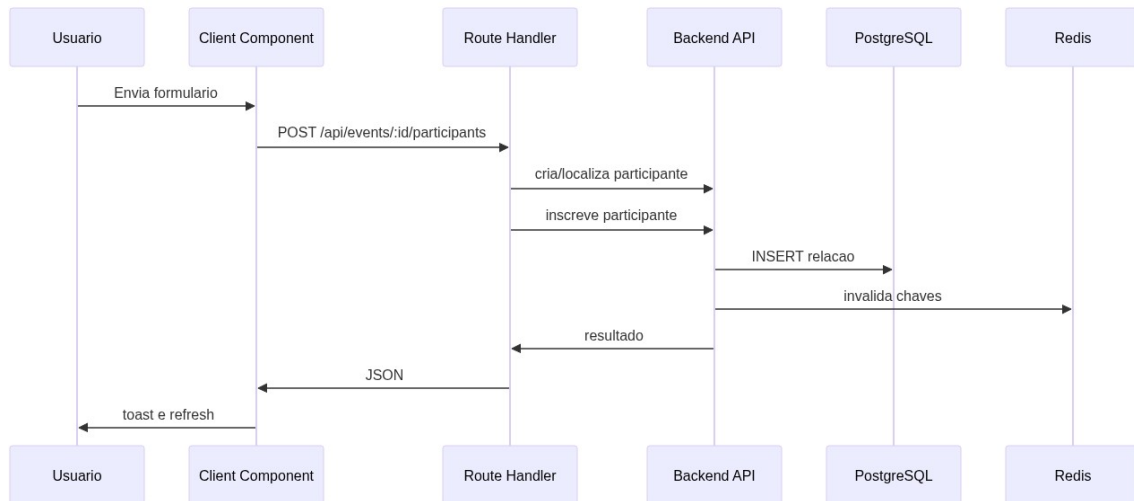
Fluxo de leitura

Exemplo: listar participantes de um evento.



Fluxo de escrita

Exemplo: inscrever participante pelo frontend.



Camadas do backend

O backend usa camadas explícitas:

- routes;
- middlewares de validacao;
- controllers;
- services;
- repositories;
- infrastructure;
- shared.

Essa estrutura reduz acoplamento entre Express, regra de negocio e Prisma.

Camadas do frontend

O frontend usa:

- pages e layouts;
- Server Components;
- Client Components;
- Route Handlers;

- services;
- hooks;
- utils;
- components compartilhados.

Essa estrutura evita que regra de chamada HTTP fique espalhada em componentes.

Infraestrutura local

`docker-compose.yml` na raiz sobe:

- PostgreSQL;
- Redis;
- backend;
- frontend.

Também permite subir apenas backend ou apenas frontend, conforme a necessidade.

Infraestrutura de produção

O deploy em VPS usa outro compose em `deploy/docker-compose.vps.yml`.

Nesse modelo:

- imagens são buildadas no GitHub Actions;
- imagens são publicadas no GHCR;
- VPS apenas faz pull e sobe containers;
- nginx publica domínios;
- certbot gerencia HTTPS.

Pontos de extensão

A arquitetura permite evoluir com:

- autenticação de usuário;

- edição de eventos;
- filtros avançados;
- auditoria;
- dashboard;
- multi-tenant;
- filas ou jobs;
- novas telas ou módulos.

Essas extensões não foram implementadas porque o case pede uma solução simples.

Backend em Alto Nível

Objetivo deste capítulo

Este capítulo resume o backend para a documentação geral. Os detalhes completos ficam em [backend/docs](#).

Stack

Area	Tecnologia
Runtime	Node.js 22
Linguagem	TypeScript strict
HTTP	Express 5
ORM	Prisma 7
Banco	PostgreSQL
Cache	Redis com ioredis
Validação	Zod

Logs	Pino
Docs	Swagger/OpenAPI
Testes	Vitest e Supertest

Modulos

O backend possui dois modulos principais:

- `events`;
- `participants`.

O modulo `events` tambem concentra inscricao e listagem de participantes de um evento, porque essa relacao nasce a partir de um evento.

Estrutura

```
backend/src/ infrastructure/ cache/ database/ modules/ events/ participants/ shared/  
index.ts server.ts
```

Principais endpoints

Eventos:

- `POST /events`;
- `GET /events`;
- `GET /events/:eventId`;
- `DELETE /events/:eventId`;
- `POST /events/:eventId/participants`;
- `GET /events/:eventId/participants`.

Participantes:

- `POST /participants`;

- `GET /participants;`
- `DELETE /participants/:participantId.`

Operação:

- `GET /health;`
- `GET /livez;`
- `GET /readyz;`
- `GET /docs;`
- `GET /docs.json.`

Banco e migrations

As migrations ficam em:

```
backend/prisma/migrations
```

Modelos principais:

- `Event;`
- `Participant;`
- `EventParticipant.`

`EventParticipant` usa chave composta para impedir que o mesmo participante seja inscrito duas vezes no mesmo evento.

Cache opcional

Redis é usado para cache de leituras, mas não é obrigatório para a API funcionar.

Se `REDIS_URL` estiver vazia ou Redis estiver indisponível, o backend segue sem cache.

Swagger

Swagger fica disponível em:

```
http://localhost:3333/docs http://localhost:3333/docs.json
```

A documentacao e modular, com blocos por modulo em:

```
backend/src/modules/*/docs
```

Seguranca

O backend usa:

- Bearer token;
- Helmet;
- CORS allowlist;
- rate limit;
- body limit;
- request id;
- erros padronizados.

Testes

O backend possui:

- testes unitarios;
- testes HTTP com Supertest;
- testes de banco real;
- testes de cache;
- testes de seed;
- testes de scripts de performance;
- testes de Swagger.

Documentacao especifica

Para detalhes internos, consulte:

[backend/docs/](#)

Frontend em Alto Nivel

Objetivo deste capitulo

Este capitulo resume o frontend para a documentacao geral. Os detalhes completos ficam em [frontend/docs](#).

Stack

Area	Tecnologia
Framework	Next.js 16
UI	React 19
Linguagem	TypeScript strict
Estilo	Tailwind CSS v4
HTTP	Axios
Datas	dayjs
Testes	Vitest e Testing Library

Rotas

Rota	Finalidade
/	Home com orientacao e proximos eventos
/events	Listagem paginada de eventos

<code>/events/new</code>	Criação de evento com prévia
<code>/events/:eventId</code>	Detalhe, participantes, busca, inscrição e delete

Estrutura

```
frontend/src/ app/ components/ config/ hooks/ lib/ services/ types/ utils/
```

Server Components

As leituras principais são feitas em Server Components. Isso protege o `API_TOKEN`, reduz chamadas no browser e deixa a página chegar com dados.

Route Handlers

As escritas feitas pelo browser passam por Route Handlers:

- `POST /api/events;`
- `POST /api/events/:eventId/participants;`
- `DELETE /api/participants/:participantId.`

Essas rotas chamam os services no lado servidor e injetam o token.

Design system

O design system é próprio, sem Material UI, Bootstrap ou bibliotecas similares.

Princípios visuais:

- base tinta sobre papel;
- ambar/mel como acento de marca;
- verde e vermelho apenas para feedback;
- componentes planos, com borda;
- foco visível;
- estados de loading, erro e vazio.

Estado

Não foi usado Redux porque o projeto não precisa de estado global complexo.

Context API foi usada para toasts. Formulários usam hook próprio `useForm`.

Paginacao e busca

O frontend pagina:

- eventos em `/events`;
- participantes em `/events/:eventId`.

Na tela de detalhe, a busca filtra participantes por:

- nome;
- e-mail;
- telefone.

Testes

Os testes cobrem:

- validações;
- mascaras;
- datas;
- extração de erro;
- formulario de evento;
- lista de participantes;
- busca;
- paginação.

Documentacao especifica

Para detalhes internos, consulte:

```
frontend/docs/
```

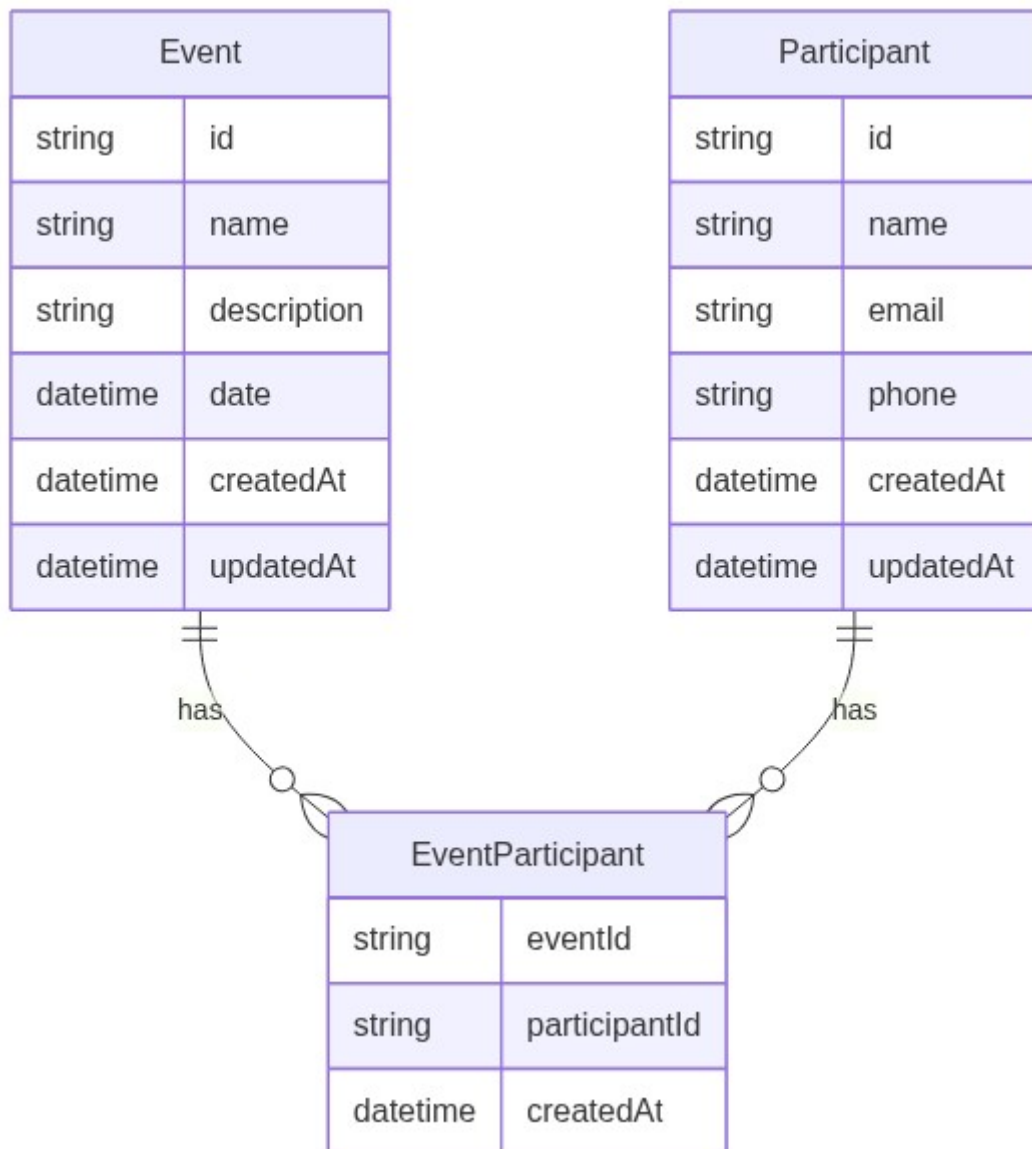
Dados, Cache e Integracoes

Objetivo deste capitulo

Este capitulo descreve como dados fluem pelo projeto, como o banco foi modelado, onde o cache entra e quais integracoes existem entre as partes.

Modelo de dados

O sistema possui tres entidades principais:



PostgreSQL

PostgreSQL foi usado por ser requisito do teste e por representar bem a relação muitos-para-muitos entre eventos e participantes.

As migrations são versionadas pelo Prisma e ficam em:

```
backend/prisma/migrations
```

Prisma

Prisma é usado para:

- schema;
- migrations;
- client tipado;
- consultas com selects explícitos.

O projeto evita buscar todos os campos automaticamente quando a consulta precisa de um formato específico.

Seed

O seed usa dados fake para facilitar teste manual e demonstração.

Ele cria:

- eventos de exemplo;
- participantes de exemplo;
- um evento com 1000 participantes.

Esse volume ajuda a validar paginação e performance.

Cache Redis

Redis cacheia respostas de leitura, como:

- listagem de eventos;
- detalhe de evento;
- listagem de participantes.

O cache tem TTL configurável por:

```
CACHE_TTL_SECONDS=60
```

Cache opcional

Redis não é dependência obrigatória para a API funcionar.

Se Redis falhar, a aplicação segue consultando o banco. Essa decisão favorece resiliência.

Invalidação

Operações de escrita invalidam chaves relacionadas:

- criação de evento;
- exclusão de evento;
- criação de participante;
- exclusão de participante;
- inscrição de participante em evento.

Integração backend-frontend

O frontend usa Axios para chamar a API. O token fica server-side.

Leituras:

- Server Components chamam services;
- services chamam `httpClient`;
- `httpClient` chama backend.

Escritas:

- Client Components chamam `/api` do Next;
- Route Handlers chamam services;
- services chamam backend.

Integração Swagger

Swagger documenta a API do backend em:

```
/docs /docs.json
```

Essa integração permite testar contratos sem depender do frontend.

Integração Docker

No compose local:

- backend acessa Postgres por `postgres:5432`;
- backend acessa Redis por `redis:6379`;
- frontend acessa backend por `http://backend:3333` quando em container.

No host local, as portas ficam expostas para facilitar desenvolvimento.

Segurança, Validação e Erros

Objetivo deste capítulo

Este capítulo resume as decisões de segurança, validação e tratamento de erros no projeto completo.

Autenticação da API

As rotas de domínio do backend usam Bearer token simples:

```
Authorization: Bearer <API_TOKEN>
```

Esse modelo atende o case sem introduzir login, sessão ou complexidade de permissões.

Token no frontend

O frontend nunca expõe `API_TOKEN` ao browser.

O token é usado apenas:

- em Server Components;
- em Route Handlers;
- no `httpClient` server-side.

Variáveis públicas `NEXT_PUBLIC_*` não são usadas para token.

Middlewares de seguranca

O backend usa:

- Helmet;
- CORS com allowlist;
- rate limit;
- body limit;
- request id.

Validacao no backend

Zod valida:

- body;
- params;
- query params;
- envs.

A validacao ocorre em middleware de rota, antes do controller.

Validacao no frontend

O frontend valida formularios para feedback imediato:

- campos obrigatorios;
- e-mail;
- telefone;
- nome;
- data;
- descricao.

Essa validacao melhora experiencia, mas o backend continua sendo a fonte final de validacao.

Erros padronizados na API

O backend responde erros no formato:

```
{ "success": false, "message": "Mensagem do erro", "error": { "code": 400, "details": {} }, "timestamp": "2026-06-09T12:00:00.000Z" }
```

Status principais:

- **400**: validacao;
- **401**: token ausente ou invalido;
- **404**: recurso inexistente;
- **409**: conflito, como e-mail ou inscricao duplicada;
- **500**: erro interno.

Tratamento de erros Prisma

Erros conhecidos do Prisma são mapeados para erros HTTP adequados. Isso evita vaziar detalhes internos do banco e melhora a previsibilidade da API.

Tratamento de erros no frontend

O frontend usa:

- `extractErrorMessage`;
- `extractErrorStatus`;
- `toasts`;
- `error.tsx`;
- `not-found.tsx`.

Com isso, erros de formulário aparecem como feedback direto e erros de carregamento aparecem em estados de rota.

Rate limit e performance

O rate limit protege a API em uso normal. Para testes de performance, é possível ajustar `RATE_LIMIT_MAX` no ambiente.

CORS

Em desenvolvimento, a allowlist inclui o frontend local.

Em deploy, `ALLOWED_ORIGINS` deve apontar para o domínio real do frontend.

Fora do escopo de segurança

Não foram implementados:

- login de usuário;
- refresh token;
- RBAC;
- OAuth;
- auditoria de usuário final.

Esses itens foram deixados fora porque o teste não pede autenticação de usuários.

Execução Local e Docker

Objetivo deste capítulo

Este capítulo mostra como rodar o projeto completo localmente, com e sem Docker, e como validar os principais fluxos.

Pre-requisitos

- Node.js 22 ou superior;
- npm;
- Docker e Docker Compose;
- Git.

Executar tudo com Docker

Na raiz:

```
docker compose up --build
```

URLs:

- frontend: <http://localhost:3000>;
- backend: <http://localhost:3333>;
- Swagger: <http://localhost:3333/docs>;
- health: <http://localhost:3333/health>.

Executar infraestrutura local

Para subir apenas PostgreSQL e Redis:

```
docker compose up -d postgres redis
```

Rodar backend localmente

```
cd backend cp .env.example .env.dev npm install npm run db:generate npm run db:migrate  
npm run db:seed npm run dev
```

No PowerShell:

```
cd backend Copy-Item .env.example .env.dev npm install npm run db:generate npm run  
db:migrate npm run db:seed npm run dev
```

Rodar frontend localmente

Com o backend no ar:

```
cd frontend cp .env.example .env.local npm install npm run dev
```

No PowerShell:


```
cd frontend Copy-Item .env.example .env.local npm install npm run dev
```

Variáveis principais

Backend:

```
PORT=3333 BASE_URL=http://localhost:3333 API_TOKEN=case-eventos-dev-token  
DATABASE_URL=postgresql://case_eventos:case_eventos@localhost:5432/case_eventos?schema=public  
REDIS_URL=redis://localhost:6379 CACHE_TTL_SECONDS=60
```

Frontend:

```
API_URL=http://localhost:3333 API_TOKEN=case-eventos-dev-token
```

Rodar somente backend via Docker

```
docker compose up --build backend
```

Rodar somente frontend via Docker

Com backend fora do compose:

```
FRONTEND_API_URL=http://host.docker.internal:3333 docker compose up --build frontend
```

No PowerShell:

```
$env:FRONTEND_API_URL="http://host.docker.internal:3333" docker compose up --build  
frontend
```

Validação manual

1. Abrir <http://localhost:3000/events>.
2. Abrir <http://localhost:3333/docs>.
3. Criar evento pelo frontend.
4. Criar participante pelo detalhe do evento.

5. Buscar participante.
6. Paginar participantes.
7. Excluir participante.
8. Consultar health do backend.

Comandos de validação

Backend:

```
cd backend npm run lint npm run typecheck npm test npm run test:database npm run build
```

Frontend:

```
cd frontend npm run lint npm run typecheck npm test npm run build
```

Observação sobre Redis

Redis é opcional no código do backend. Para rodar sem cache, deixe `REDIS_URL` vazio no env do backend.

Deploy, CI/CD e Operação

Objetivo deste capítulo

Este capítulo descreve o fluxo de CI/CD e deploy em VPS, incluindo GitHub Actions, GHCR, Docker Compose, nginx e certbot.

Workflows

O projeto possui dois workflows:

```
.github/workflows/ci.yml .github/workflows/deploy.yml
```

CI

O CI valida backend e frontend em pushes e pull requests para `development` e `main`.

O objetivo é impedir que código sem teste, lint, typecheck ou build válido avance para deploy.

Deploy

O deploy:

1. builda imagens Docker no GitHub Actions;
2. publica imagens no GHCR;
3. conecta na VPS por SSH;
4. prepara arquivos de deploy;
5. faz pull das imagens;
6. sobe containers via Docker Compose;
7. configura nginx;
8. emite/renova certificados com certbot quando necessário.

Ambientes

Branch `development`:

- Environment GitHub: `development`;
- domínio de desenvolvimento;
- compose project name próprio;
- portas próprias.

Branch `main`:

- Environment GitHub: `production`;
- domínio de produção;
- compose project name próprio;
- portas próprias.

Imagens

Formato das imagens:

```
ghcr.io/cledson96/case-eventos-backend:<sha>  
ghcr.io/cledson96/case-eventos-frontend:<sha>
```

Arquivos de deploy

```
deploy/ docker-compose.vps.yml nginx/ reverse-proxy-http.conf reverse-proxy-https.conf  
scripts/ deploy-docker.sh
```

Secrets obrigatórios

```
VPS_HOST VPS_USER VPS_SSH_KEY API_TOKEN POSTGRES_PASSWORD
```

Secrets opcionais

```
VPS_PORT CERTBOT_EMAIL DATABASE_URL GHCR_PULL_TOKEN
```

Vars obrigatórias

```
FRONTEND_DOMAIN BACKEND_DOMAIN
```

Vars recomendadas

```
DEPLOY_PATH=/opt/case-eventos-development  
COMPOSE_PROJECT_NAME=case-eventos-development FRONTEND_PORT=3001 BACKEND_PORT=3334  
BASE_URL=https://api-eventos-dev.seudominio.com  
FRONTEND_PUBLIC_URL=https://eventos-dev.seudominio.com  
ALLOWED_ORIGINS=https://eventos-dev.seudominio.com RUN_SEED_ON_DEPLOY=true
```

Seed em deploy

`RUN_SEED_ON_DEPLOY=true` popula o ambiente com dados fake.

Essa opção é útil em ambiente de desenvolvimento/demo. Em produção real, o mais seguro é deixar `false`, a menos que o objetivo seja demonstração.

nginx e certbot

nginx fica no host e publica os domínios do frontend e backend. Certbot também roda no host.

Os containers não precisam gerenciar TLS diretamente.

Banco em VPS

No compose de VPS, o Postgres deve ficar dentro da rede Docker, sem exposição pública direta. O acesso externo ao banco não é necessário para o uso da aplicação.

Validação pos-deploy

Depois do deploy:

1. abrir domínio do frontend;
2. abrir Swagger do backend;
3. validar `/health`;
4. criar evento;
5. inscrever participante;
6. verificar logs dos containers;
7. validar certificados HTTPS.

Comandos úteis na VPS

```
docker compose ps docker compose logs -f backend docker compose logs -f frontend
docker compose pull docker compose up -d sudo nginx -t sudo systemctl reload nginx
```

Testes, Qualidade e Performance

Objetivo deste capítulo

Este capitulo resume como o projeto e validado em qualidade, testes, build e performance.

Qualidade geral

O projeto usa:

- TypeScript strict;
- ESLint;
- Prettier;
- Vitest;
- Supertest;
- Testing Library;
- migrations reais;
- Docker build;
- CI no GitHub Actions.

Testes do backend

Comando principal:

```
cd backend npm test
```

Cobre:

- services;
- repositories;
- schemas;
- cache;
- HTTP com Supertest;
- Swagger;
- seed;
- scripts auxiliares.

Testes de banco

```
cd backend npm run test:database
```

Esse comando aplica migrations em banco de teste e valida comportamento com PostgreSQL real.

Testes do frontend

```
cd frontend npm test
```

Cobre:

- formularios;
- validacoes;
- mascaras;
- datas;
- erros;
- paginacao;
- lista e busca de participantes.

Typecheck

Backend:

```
cd backend npm run typecheck
```

Frontend:

```
cd frontend npm run typecheck
```

Lint

Backend:

```
cd backend npm run lint
```

Frontend:

```
cd frontend npm run lint
```

Build

Backend:

```
cd backend npm run build
```

Frontend:

```
cd frontend npm run build
```

Performance backend

O backend possui scripts para testar carga no endpoint de participantes do evento seedado.

Teste pontual:

```
cd backend npm run perf:event-participants
```

Sweep de conexões:

```
npm run perf:event-participants:sweep
```

Variáveis úteis:

```
PERF_BASE_URL=http://localhost:3333 PERF_CONNECTIONS=10 PERF_DURATION_SECONDS=10  
PERF_LIMIT=100 PERF_PIPELINING=1 PERF_SWEEP_CONNECTIONS=5,10,25,50,100
```

Performance frontend

O frontend evita carregar listas grandes sem paginação:

- eventos usam `limit=12`;
- participantes usam `limit=10`;
- busca preserva query e pagina;
- textos longos usam truncamento;
- leituras principais são server-side.

Validação final recomendada

Na raiz, após subir dependências:

```
cd backend npm run lint npm run typecheck npm test npm run test:database npm run build  
cd ../frontend npm run lint npm run typecheck npm test npm run build
```

Evidências de maturidade

Pontos que demonstram cuidado técnico:

- testes em mais de uma camada;
- banco real em suite dedicada;
- Swagger protegido contra regressão de build;
- cache opcional;
- Docker local e VPS;
- documentação técnica;
- commits organizados.

Decisões Técnicas e Tradeoffs

Objetivo deste capítulo

Este capítulo registra decisões importantes do projeto e os tradeoffs envolvidos. A ideia é mostrar que as escolhas foram proporcionais ao escopo do case.

Monorepo

O projeto usa monorepo com **backend** e **frontend**.

Vantagens:

- facilita avaliacao;
- facilita docker compose local;
- centraliza deploy;
- evita repos separados para um case pequeno.

Tradeoff:

- pipelines precisam separar jobs por pasta.

Express em vez de framework maior

Express foi suficiente para o escopo REST do teste.

Vantagens:

- simples;
- conhecido;
- compativel com Supertest e Swagger;
- pouco overhead.

Tradeoff:

- exige organizar manualmente camadas e padroes.

Prisma

Prisma foi escolhido para ORM e migrations.

Vantagens:

- schema claro;
- client tipado;
- migrations versionadas;

- boa experiência com TypeScript.

Tradeoff:

- adiciona geração de cliente e cuidado com versões.

Redis opcional

Redis melhora leituras, mas a API não depende dele.

Vantagens:

- melhora performance;
- demonstra infraestrutura;
- não derruba a API se falhar.

Tradeoff:

- adiciona complexidade de invalidação.

Bearer token simples

Foi usado token simples em vez de login.

Vantagens:

- protege rotas de domínio;
- atende o case sem escopo extra;
- simples para Swagger, frontend e deploy.

Tradeoff:

- não representa usuários reais nem permissões por perfil.

Next.js com BFF

O frontend usa Route Handlers como BFF para escritas.

Vantagens:

- token não fica no browser;
- erros podem ser normalizados;
- componentes client-side continuam simples.

Tradeoff:

- existe uma camada a mais entre formulário e backend.

Context API em vez de Redux

Redux não foi usado porque não havia estado global complexo.

Vantagens:

- menos dependência;
- menos boilerplate;
- estado de servidor fica no servidor.

Tradeoff:

- se a aplicação crescer muito, pode ser necessário reavaliar.

Tailwind sem biblioteca de UI

Tailwind foi usado para criar componentes próprios.

Vantagens:

- atende requisito de não usar Material UI ou Bootstrap;
- permite identidade visual própria;
- reduz dependência de componentes externos.

Tradeoff:

- exige construir estados e componentes manualmente.

Paginacao no frontend

Listagens usam paginacao mesmo quando o case minimo nao exige.

Vantagens:

- escala com dados maiores;
- combina com seed de 1000 participantes;
- reduz peso da UI.

Tradeoff:

- adiciona query params e navegacao.

Seed com volume alto

O seed cria um evento com 1000 participantes.

Vantagens:

- facilita demonstracao;
- valida paginacao;
- permite teste de performance.

Tradeoff:

- seed pode ser pesado para ambientes pequenos se usado sem criterio.

Deploy em VPS

O projeto inclui deploy realista com Docker, GHCR, nginx e certbot.

Vantagens:

- demonstra operação;
- separa development e production;
- VPS não faz build.

Tradeoff:

- exige secrets e DNS configurados.

Decisão geral

As escolhas adicionam maturidade sem transformar o case em um produto grande. O foco continua sendo uma aplicação simples, funcional e bem organizada.

Glossário e Leitura Rápida

Objetivo deste capítulo

Este capítulo define termos usados no projeto e oferece uma trilha rápida de leitura para avaliação.

Glossário

API

Backend HTTP em Express responsável por eventos, participantes e inscrições.

BFF

Backend for Frontend. No projeto, são os Route Handlers do Next.js em `frontend/src/app/api`.

Cache

Armazenamento temporário em Redis para respostas de leitura.

CI

Pipeline de integracao continua que valida lint, typecheck, testes e build.

Deploy

Processo de publicar imagens Docker no GHCR e atualizar containers na VPS.

Docker Compose

Arquivo que orquestra Postgres, Redis, backend e frontend.

Event

Evento cadastrado no sistema.

EventParticipant

Relacao entre evento e participante. Representa inscricao.

Frontend

Aplicacao Next.js com React, TypeScript, Tailwind e Axios.

GHCR

GitHub Container Registry, usado para armazenar imagens Docker do projeto.

Migration

Arquivo versionado que altera o schema do banco.

Participant

Pessoa cadastrada para participar de eventos.

Prisma

ORM usado no backend para schema, migrations e consultas.

Route Handler

Rota HTTP interna do Next.js.

Server Component

Componente React renderizado no servidor pelo Next.js.

Swagger

Interface visual da documentacao OpenAPI do backend.

TTL

Tempo de vida de uma chave de cache.

VPS

Servidor privado usado para hospedar os containers.

Leitura rapida para avaliador

Para validar requisitos

Leia:

```
docs/02-aderencia-ao-desafio.md
```

Para entender arquitetura

Leia:

```
docs/04-arquitetura-fullstack.md backend/docs/04-arquitetura.md  
frontend/docs/04-arquitetura.md
```

Para rodar o projeto

Leia:

```
README.md docs/09-execucao-local-e-docker.md
```

Para validar qualidade

Leia:

```
docs/11-testes-qualidade-e-performance.md backend/docs/10-testes-validacao.md  
frontend/docs/10-testes-validacao.md
```

Para entender deploy

Leia:

```
docs/10-deploy-ci-cd-e-operacao.md backend/docs/09-deploy-operacao.md  
frontend/docs/09-deploy-operacao.md
```

Para entender decisoes

Leia:

```
docs/12-decisoes-tecnicas-e-tradeoffs.md
```

Resumo executivo

O Case Eventos e uma aplicacao fullstack simples, mas estruturada com padroes profissionais:

- backend REST com Express, TypeScript, Prisma e PostgreSQL;
- frontend com Next.js, React, TypeScript e Tailwind;
- migrations versionadas;
- Swagger;
- Docker;
- CI/CD;
- testes;
- cache opcional;
- documentacao completa.

O resultado atende o desafio e demonstra organizacao, clareza e boas praticas de engenharia.